



PROVA FINALE · INGEGNERIA DEL SOFTWARE · GRUPPO PSP30

Mesos

Implementazione Java di un gioco da tavolo client-server

Riccardo Libera · Francesco Leucci · Matteo Mancin · Cesare Manzini

FUNZIONALITÀ IMPLEMENTATE

✓ Regole complete

✓ TUI + GUI (JavaFX)

✓ RMI + Socket

✓ 2 Funzionalità Avanzate

Funzionalità avanzate: classifica delle partite su DB · partite multiple in contemporanea



Architettura — MVC client-server

Pattern MVC con server centralizzato; la logica di gioco è indipendente dal trasporto di rete.



model

Stato e logica di gioco: carte, board, tile, eventi, building, personaggi



controller

GameController — orchestrazione della partita e validazione delle azioni



network

db · messages · rmi · socket · snapshots — connettività e persistenza



view

gui (JavaFX) · cli (TUI) — client indipendente dal trasporto

Idea chiave. Il GameController dialoga solo con le astrazioni **VirtualView** (lato server) e **NetworkClient / ui** (lato client): RMI e Socket sono intercambiabili senza toccare la logica di gioco.



Design pattern & scelte implementative



MVC

Separazione netta tra model, controller e view.



Observer

GameObserver (Game → Controller per eventi e fine partita) e GameStarter (Controller → GameManager all'avvio).



Adapter / Proxy

VirtualView e NetworkClient astraggono il client dal trasporto (RMI o Socket).



DTO / Snapshot

BoardSnapshot e PlayerSnapshot: stato leggero e immutabile inviato in rete al posto del model.



Template method (hook)

BuildingCard: onPurchase / onRoundEnd / onCharacterCardPurchase / onGameEnd, ridefiniti per ogni edificio.



Deserializzazione polimorfica

Carte (eventi, building, personaggi) caricate da file JSON tramite Jackson.



Protocolli di comunicazione

Due trasporti, stessa logica applicativa: un unico GameController serve sia i client RMI sia i client Socket.

RMI

- ServerInterface — chiamate client → server
- ClientRemote — callback server → client (board, turno, eventi, chat, fine partita)
- Registry "MesosServer" · UnicastRemoteObject

Socket

- Messaggi serializzati (package messages) processati dal ricevente
- Stessa VirtualView lato server: nessuna duplicazione di logica
- Broadcast asincroni verso i client connessi

In comune ai due trasporti

Sessioni a token UUID (login → receiveToken → token a ogni richiesta); **GameManager**: registro centrale di sessioni e partite, distingue lobby (availableGames) e partite avviate (startedGames). **Ogni endpoint RMI/Socket** risolve token → sessione → GameController e ne invoca l'azione.

Sequence diagram di riferimento: addPlayer (lobby → avvio partita).



Test & divisione del lavoro

TEST AUTOMATICI

109/109

test verdi

100%

classi model coperte

COPERTURA

model (righe)



totale (righe)



83% righe

controller · network · view: validati con test manuali e partite reali su più dispositivi.

DIVISIONE DEL LAVORO

Area	Responsabile
Model – Core (Game , Player, Board, Tiles)	Leucci Mancin
Model – Tribe Cards (Character, Event)	Libera Manzini
Model – Buildings	Comune
Controller, GameManager, Snapshot	Manzini
Rete – RMI	Mancin
Rete – Socket	Mancin Manzini
Persistenza DB	Mancin
View – TUI	Mancin
View - GUI	Comune
Test & build	Comune